KUBERNETES TRAINING

Module 7:

Resource Management, Health Probes & Pod Scheduling



Give workloads enough resources, know when they are healthy, and control where Pods land.

2026 MODERNIZED EDITION

From containers to controlled placement

Modules 1–3

Docker, Kubernetes architecture, kubectl, Pods + Services

Modules 4–5

Namespaces, Deployments, Services, node maintenance, DaemonSets, Jobs, CronJobs

Module 6

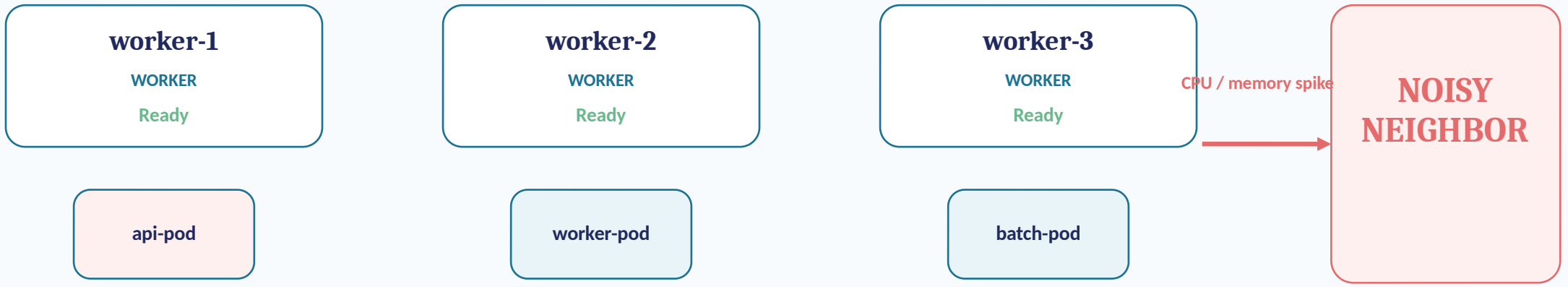
ConfigMaps, Secrets, Init Containers, Sidecar / Adapter / Ambassador

TODAY

1. Requests, limits + QoS
2. Security context: run safely
3. Liveness / readiness / startup probes
4. Node affinity + anti-affinity
5. Pod affinity + anti-affinity
6. Taints + tolerations
7. Operations lab + 2026 scheduling updates

Why resource management matters

Without requests and limits, one noisy Pod can consume a node's headroom and starve neighboring workloads.



Requests reserve scheduling capacity • Limits cap runtime consumption • Together they make placement and runtime behavior predictable.

Requests vs Limits — floor vs ceiling

REQUEST = SCHEDULING FLOOR

The minimum CPU / memory the scheduler uses when deciding whether a Node has enough capacity.

`spec.resources.requests • 500m CPU • 256Mi memory`

LIMIT = RUNTIME CEILING

The maximum CPU / memory the container can consume. CPU can throttle; memory overage can trigger OOM kill.

`spec.resources.limits • 1 CPU • 512Mi memory`

Resource Limits & Requests — full Pod manifest

resource-limited-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-limited-pod
spec:
  containers:
    - name: busybox
      image: busybox:1.37
      command: ["sh", "-c", "while true; do echo Running...; sleep
5; done"]
      resources:
        requests:
          cpu: "500m"
          memory: "256Mi"
        limits:
          cpu: "1"
          memory: "512Mi"
```

$ Terminal

```
kubectl apply -f resource-
limited-pod.yaml
```

OUTPUT

```
pod/resource-limited-pod created
```

$ Terminal

```
kubectl get pod resource-limited-
pod
```

OUTPUT

```
resource-limited-pod   1/1   Running
```

Verify what Kubernetes accepted

$ Terminal

```
kubectl describe pod resource-
limited-pod
```

Read the Requests/Limits block

Requests are used for scheduling. Limits describe the container ceiling enforced at runtime.

$ Terminal

```
kubectl get pod resource-limited-pod -o yaml
```

OUTPUT

```
Name:         resource-limited-pod
Containers:
  busybox:
    Limits:
      cpu:     1
      memory:  512Mi
    Requests:
      cpu:     500m
      memory:  256Mi
QoS Class:    Burstable
```

The Pod spec is the source of truth for the declared values.

Monitor live usage with kubectl top

$ Terminal

```
kubectl top pod resource-limited-pod
```

OUTPUT

```
NAME                   CPU(cores)   MEMORY(bytes)
resource-limited-pod   1m           0Mi
```

Metrics prerequisite

`kubectl top` reads the Resource Metrics API (`metrics.k8s.io`). Kubernetes doesn't serve it itself — install metrics-server (Module 5, Step 17).

Check it: kubectl api-versions | grep metrics → metrics.k8s.io/v1beta1 (metrics-server v0.9.0 on Kubernetes v1.37).

When limits are exceeded: CPU vs memory

CPU > LIMIT

Container keeps running, but CPU is throttled.



busy → throttled → slower work

MEMORY > LIMIT

The container can be OOMKilled; the Pod may restart it according to restart policy.



over limit → OOMKilled → restart

Interview / exam distinction: CPU limit = throttle • memory limit = kill

QoS classes — the label Kubernetes gives every Pod

You never set the class. Kubernetes reads your requests and limits and decides it for you.

Guaranteed

EVICTED LAST

Every container has CPU and memory requests, and requests = limits.

```
resources:
  requests:
    cpu: 100m
    memory: 64Mi
  limits:
    cpu: 100m
    memory: 64Mi
```

For: databases, payments, anything critical.

Burstable

IN BETWEEN

At least one request or limit is set, but it is not Guaranteed.

```
resources:
  requests:
    cpu: 100m
    memory: 64Mi
  limits:
    cpu: 500m
    memory: 128Mi
```

For: most normal apps. The usual choice.

BestEffort

EVICTED FIRST

No requests and no limits at all. Nothing is promised to the Pod.

```
# no resources: block
# at all
```

For: throw-away or test Pods only.

Check any Pod: kubectl get pod NAME -o jsonpath='{.status.qosClass}' • Hands-on: slide 38, demo Step 3

⚠ Node out of memory — who is evicted first

When a node runs short of memory, Kubernetes has to stop some Pods to save the node. The QoS class decides the order.

1 BestEffort
FIRST
Nothing was promised, so these go first.

→

2 Burstable
NEXT
Pods using more than they requested go first.

→

3 Guaranteed
LAST
Not evicted because of another Pod's memory use.

How the kubelet chooses between Pods

1 Pods using more memory than they requested go first.

2 Among those, the lower Pod priority goes first.

3 Then the Pod furthest above its request.

A Burstable Pod that stays under its request is safe until almost the end.

Evicted is not the same as OOMKilled

Evicted: the kubelet stops the Pod in time to protect the node. A Deployment makes a new Pod on another node.

OOMKilled: the kernel kills a container that crossed its own memory limit, or when the node is fully out. It also picks BestEffort, then Burstable, before Guaranteed.

CPU: too much CPU only slows Pods down. Nothing is evicted for CPU.

Rule of thumb: critical Pods → Guaranteed • normal apps → Burstable • BestEffort only for throw-away Pods

Containers run as root by default

securityContext is a few lines in the Pod file that say what the container is allowed to do.

Default: no securityContext

Runs as root.

Can change any file inside the container.

Keeps extra Linux powers that it rarely needs.

If the app is hacked, the attacker gets all of this.

With securityContext

Runs as a normal user.

Cannot change its own files.

No extra powers.

If the app is hacked, the damage stays small.

Resources say HOW MUCH a container may use. securityContext says WHAT it is allowed to do.

Three simple rules for a safe container

You only need to remember three rules.

1 Don't run as root

```
runAsNonRoot: true
runAsUser: 1000
runAsGroup: 3000
```

The container runs as a normal user and group, not as root.

2 Don't change files

```
readOnlyRootFilesystem: true
```

The container cannot change its own files. If it must write, give it a temporary folder (emptyDir).

3 Don't gain extra powers

```
allowPrivilegeEscalation: false
capabilities:
  drop: ["ALL"]
```

The container gets no extra Linux powers, and cannot get them later.

Volume says "permission denied"? **fsGroup: 2000** gives files in a mounted volume this group, so the app can write there.

Not sure what to pick? Use all three rules. Most apps work fine with them.

A safe Pod you can copy

Start every app from this. Change a line only when the app really needs it.

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: myapp:1.0
      securityContext:
        readOnlyRootFilesystem: true
        allowPrivilegeEscalation: false
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - { name: tmp, mountPath: /tmp }
  volumes:
    - name: tmp
      emptyDir: {}
```

Where does each setting go?

Pod level (under spec → securityContext)

runAsNonRoot, runAsUser, runAsGroup, fsGroup, seccompProfile

Container level (inside the container)

readOnlyRootFilesystem, allowPrivilegeEscalation, capabilities

Both set? The container wins. **Must write?** Mount an emptyDir.

Module 9 makes Kubernetes enforce this for a whole namespace. Lab: slide 40, demo Steps 7–9.

What are Kubernetes probes?

Probes are periodic health checks used by the kubelet to decide whether a container is alive, ready for traffic, or still starting.

STARTUP

Has boot completed?

Protect slow starts

LIVENESS

Is it still healthy?

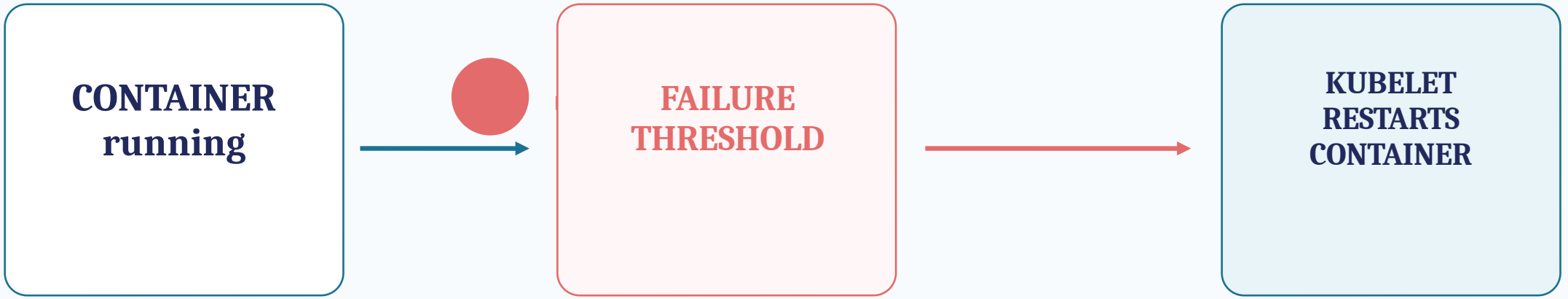
Restart when stuck

READINESS

Can it serve traffic?

Remove from endpoints

Liveness Probe — recover a broken container



Use liveness for

Deadlocks, crashes, or a process that is alive but no longer healthy. A failed liveness probe is a restart signal.

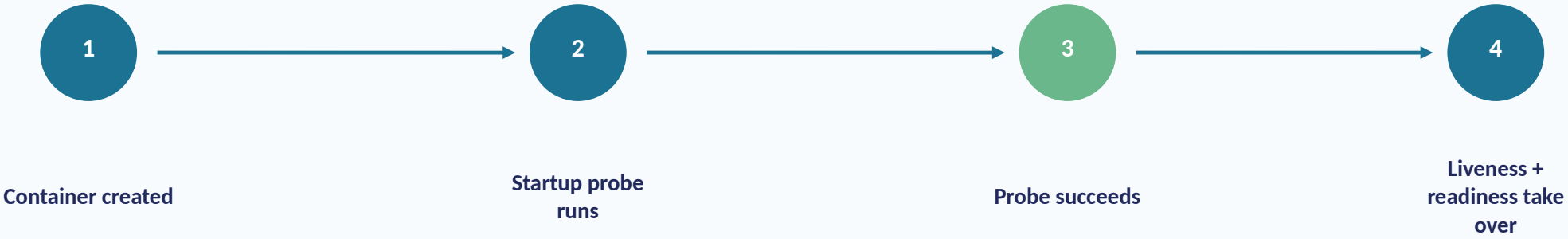
Readiness Probe — control traffic, not lifecycle



Container keeps running. No restart is implied by a readiness failure.

Think: “Can I receive traffic?” — not “Should I be restarted?”

Startup Probe — protect slow application boots



Startup failure beyond its threshold causes a restart; until startup succeeds, liveness/readiness are not allowed to prematurely judge the app.

The three-probe mental model

STARTUP

“Did you finish booting?”

Runs first for slow-start protection

Failure → restart

LIVENESS

“Are you still healthy?”

Runs through lifecycle

Failure → restart

READINESS

“Can you take traffic?”

Runs through lifecycle

Failure → remove from Service endpoints



Probes YAML — all three in one Pod

probes-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: probe-demo
spec:
  terminationGracePeriodSeconds: 5   # sh ignores SIGTERM — don't wait the default 30 s
  containers:
    - name: my-app
      image: busybox:1.37
      # ready after 30 s, "breaks" 30 s later by deleting the file
      command: ["/bin/sh", "-c", "sleep 30 && touch /tmp/ready && sleep 30 && rm -f /tmp/ready &&
sleep 600"]
      startupProbe:
        exec:
          command: ["cat", "/tmp/ready"]
        periodSeconds: 5
        failureThreshold: 12        # up to 60 s to finish booting
      readinessProbe:
        exec:
          command: ["cat", "/tmp/ready"]
        periodSeconds: 3
        failureThreshold: 2
      livenessProbe:
        exec:
          command: ["cat", "/tmp/ready"]
        periodSeconds: 5
        failureThreshold: 3
```

$ Terminal

```
kubectl apply -f probes-pod.yaml
kubectl get pods
```

OUTPUT

```
NAME         READY   STATUS
probe-demo   0/1     Running

# startup still running
```

$ Terminal

```
kubectl describe pod probe-
demo
```

OUTPUT

Apply, watch, and inspect probe behavior

$ Terminal

```
kubectl apply -f probes-pod.yaml
kubectl get pod probe-demo -w
```

OUTPUT

```
probe-demo   0/1   Running   0            1s
probe-demo   1/1   Running   0            35s
probe-demo   0/1   Running   0            65s
probe-demo   0/1   Running   1 (1s ago)   81s
```

$ Terminal

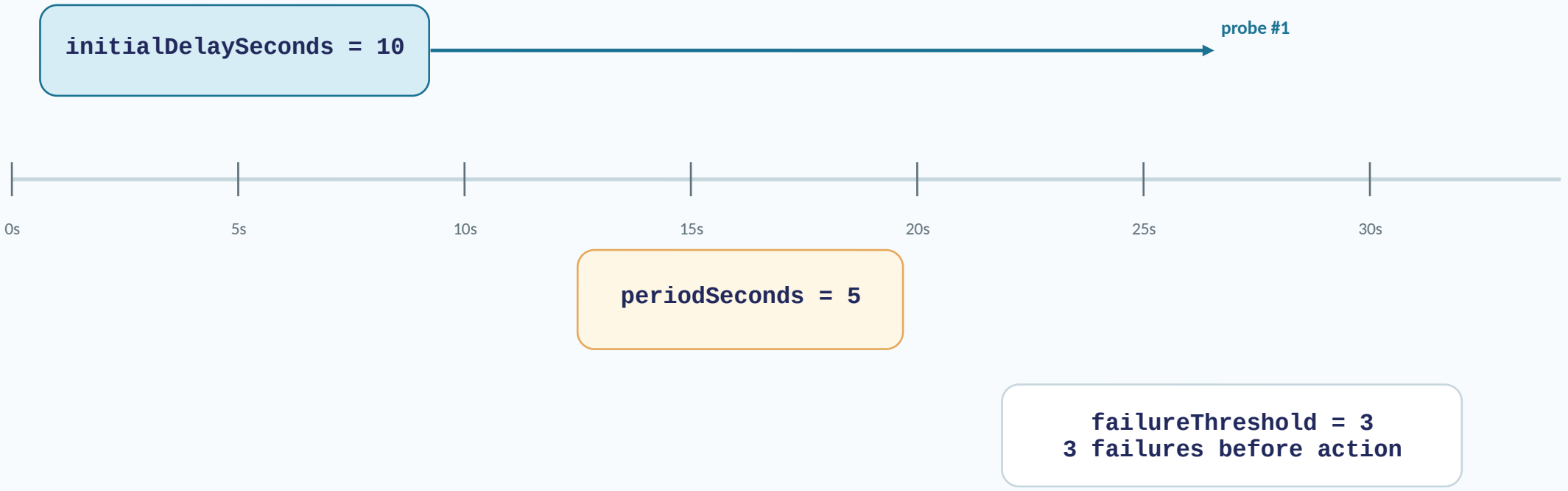
```
kubectl describe pod probe-demo
```

OUTPUT

```
Warning  Unhealthy  Startup probe failed: cat: can't open '/tmp/ready'
Warning  Unhealthy  Readiness probe failed: cat: can't open …
Warning  Unhealthy  Liveness probe failed: cat: can't open …
Normal   Killing    Container my-app failed liveness probe,
                    will be restarted
```

Watch two signals: READY changes when traffic eligibility changes; RESTARTS increments when liveness causes a restart.

Tune probe timing: detect failures without flapping



Too aggressive → false restarts / flapping
Too relaxed → slow failure detection

Why control Pod placement?

SSD

storage=ssd

fast local I/O

GPU

accelerator=nvidia

special hardware

ZONE

topology.kubernetes.io/zone

fault-domain placement

Scheduling is a placement decision: resources + constraints + labels + affinity + taints all shape where a Pod can run.

Label Nodes — the foundation for affinity

$ Terminal

```
kubectl label node lab-worker2 storage=ssd
kubectl label node lab-worker storage=hdd
```

OUTPUT

```
node/lab-worker2 labeled
node/lab-worker labeled
```

$ Terminal

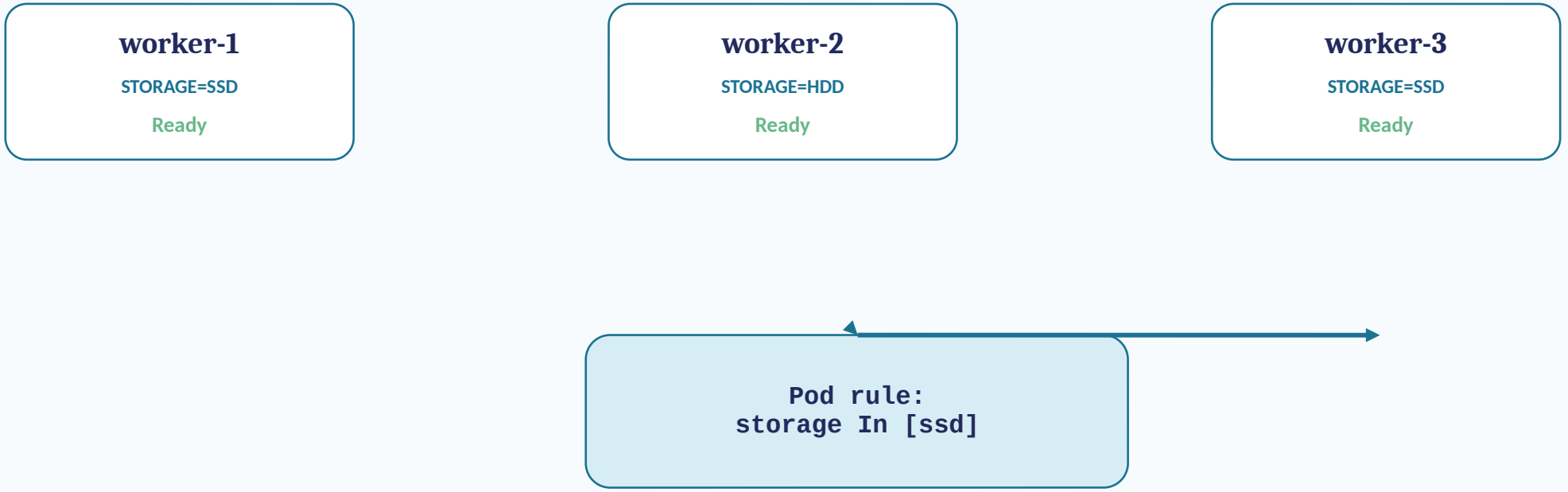
```
kubectl get nodes -L storage
```

OUTPUT

```
NAME                 STATUS   ROLES           STORAGE
lab-control-plane    Ready    control-plane
lab-worker           Ready    <none>          hdd
lab-worker2          Ready    <none>          ssd
```

Labels are data on the Node; affinity rules are the Pod-side query against that data. -L adds a label as a column.

Node Affinity — Pods choose matching Nodes



Only matching labels satisfy a required rule.

Required Node Affinity — strict placement

node-affinity-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: node-affinity-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: storage
                operator: In
                values:
                  - ssd
  containers:
    - name: nginx
      image: nginx:1.28-alpine
```

$ Terminal

```
kubectl apply -f node-affinity-pod.yaml
kubectl get pods -o wide
```

OUTPUT

```
NAME                READY   NODE
node-affinity-pod   1/1     lab-
worker2
```

Why it landed there

Only lab-worker2 carries storage=ssd. With no match the Pod stays Pending — and it isn't evicted if the label is removed later.

Required vs Preferred Affinity

REQUIRED

Hard constraint

No matching Node → Pod stays Pending

```
requiredDuringSchedulingIgnoredDuringExecution
```

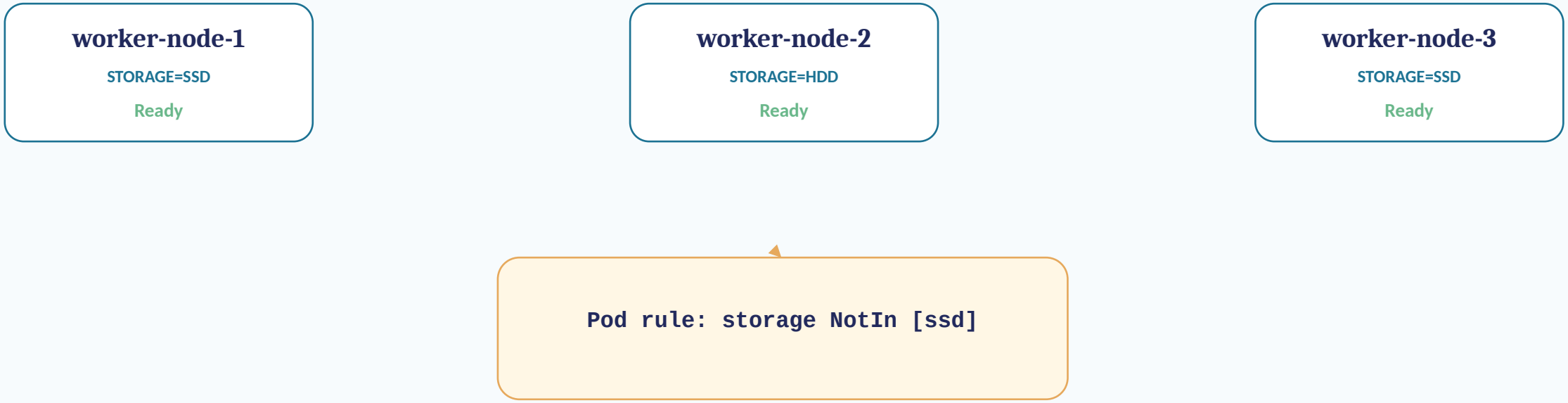
PREFERRED

Soft preference

No match → scheduler may place elsewhere

```
preferredDuringSchedulingIgnoredDuringExecution
```

"Node anti-affinity" — NotIn and DoesNotExist



No nodeAntiAffinity field exists: write NotIn / DoesNotExist in nodeAffinity, or taint the node.

Pod Affinity & Anti-Affinity — relate Pods to Pods

POD AFFINITY • podAffinity



"Place me with app=frontend"

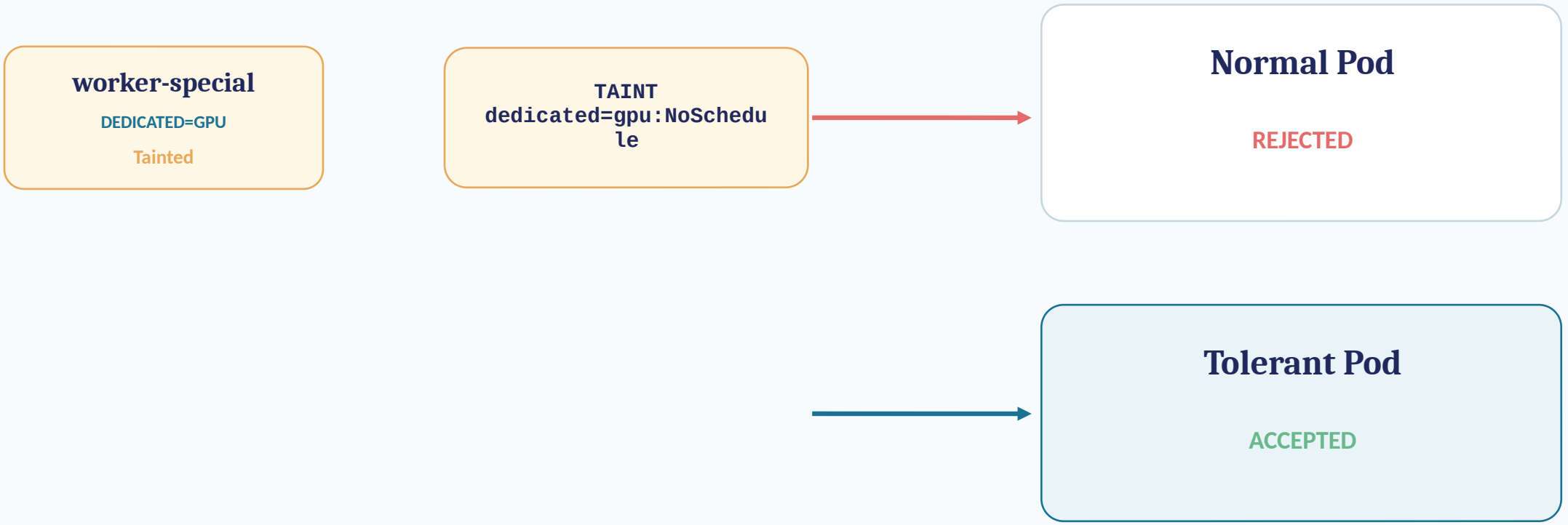
POD ANTI-AFFINITY • podAntiAffinity



"Keep app=database apart"

Use topologyKey to define the domain: hostname, zone, or another topology boundary.

Taints & Tolerations — Nodes repel, Pods tolerate



⚠ Taint effects — three distinct behaviors

EFFECT	NEW PODS	EXISTING PODS	MENTAL MODEL
NoSchedule	Blocked unless tolerated	Keep running	**“No new arrivals”**
PreferNoSchedule	Avoid if possible	Keep running	**“Try elsewhere”**
NoExecute	Blocked unless tolerated	Evict if not tolerated	**“Clear the node”**

Apply a NoSchedule taint and inspect it

$ Terminal

```
kubectl taint nodes lab-worker2 \
  dedicated=ops:NoSchedule
```

OUTPUT

```
node/lab-worker2 tainted
```

$ Terminal

```
kubectl describe node lab-worker2 \
  | grep Taints
```

OUTPUT

```
Taints:             dedicated=ops:NoSchedule
```

NoSchedule blocks new Pods without a matching toleration. Pods already running on the node are not evicted by this effect.

Test the taint — a normal Pod stays Pending

$ Terminal

```
kubectl apply -f taint-pods.yaml
kubectl describe pod normal-pod
```

OUTPUT

```
Name:         normal-pod
Status:       Pending
Events:
  Type     Reason            Message
  Warning  FailedScheduling  0/3 nodes are available:
    1 node(s) didn't match Pod's node affinity/selector,
    2 node(s) had untolerated taint(s).
```

normal-pod is pinned to lab-worker2 with nodeSelector. Unpinned, it would simply run on another node; only when no node fits does it stay Pending.

Add a matching Toleration

ops-pod (from taint-pods.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: ops-pod
spec:
  nodeSelector:
    kubernetes.io/hostname: lab-worker2
  tolerations:
    - key: dedicated
      operator: Equal
      value: ops
      effect: NoSchedule
  containers:
    - name: nginx
      image: nginx:1.28-alpine
```

$ Terminal

```
kubectl apply -f taint-pods.yaml
kubectl get pod ops-pod -o wide
```

OUTPUT

```
NAME      READY   STATUS    NODE
ops-pod   1/1     Running   lab-worker2
```

Important

A toleration allows the tainted node but never attracts the Pod to it. Here nodeSelector does the attracting.

NoExecute — evict an already-running Pod

$ Terminal

```
kubectl taint nodes lab-worker2 \
  dedicated=ops:NoExecute
```

OUTPUT

```
node/lab-worker2 tainted

web (no toleration)    evicted at once
ops-pod (NoSchedule)   evicted at once
ops-agent (no effect)  keeps running
```



Remove a taint — note the trailing minus

$ Terminal

```
kubectl taint nodes lab-worker2 dedicated=ops:NoSchedule-
kubectl taint nodes lab-worker2 dedicated-
```

OUTPUT

```
node/lab-worker2 untainted
node/lab-worker2 untainted
```

The final `-` removes. key=value:Effect- removes one taint; key- removes all.

Taints are keyed by key + effect, so a node can carry dedicated=ops:NoSchedule and dedicated=ops:NoExecute at the same time.

Affinity vs Taints — the mental model

AFFINITY

"Pods choose Nodes"



Labels → rules → placement

TAINTS

"Nodes reject Pods"



Taint → toleration → admission

Use affinity to attract / require. Use taints to repel unless explicitly tolerated.

Hands-on lab — resources + probes + placement

1 **Resource floor / ceiling**

$ Terminal

```
kubectl apply -f resource-limited-pod.yaml
kubectl describe pod resource-limited-pod
```

2 **Add health checks**

$ Terminal

```
kubectl apply -f probes-pod.yaml
kubectl get pods -w
```

3 **Label a node**

$ Terminal

```
kubectl label node lab-worker2 storage=ssd
kubectl get nodes -L storage
```

4 **Pin with affinity**

$ Terminal

```
kubectl apply -f node-affinity-pod.yaml
kubectl get pods -o wide
```

5 **Repel workloads**

$ Terminal

```
kubectl taint nodes lab-worker2 \
  dedicated=ops:NoSchedule
```

6 **Verify + clean up**

$ Terminal

```
kubectl describe pod <name>
kubectl taint nodes lab-worker2 dedicated-
```

Lab: requests, limits and QoS classes

$ Terminal

```
kubectl apply -f resource-limited-pod.yaml
kubectl describe pod … | grep -A3 Requests
```

OUTPUT

```
Requests:  cpu 500m   memory 256Mi
Limits:    cpu 1      memory 512Mi
QoS Class: Burstable
```

$ Terminal

```
kubectl describe node lab-worker \
  | grep -A5 'Allocated resources'
```

OUTPUT

```
cpu     600m (15%)   1 (25%)
memory  306Mi (1%)   512Mi (3%)
# requests are booked — even when idle
```

$ Terminal

```
kubectl apply -f qos-pods.yaml
kubectl get pods -o custom-columns=…
```

OUTPUT

```
qos-guaranteed   Guaranteed  ← requests = limits
qos-burstable    Burstable
qos-besteffort   BestEffort  ← evicted first
```

Lab: throttled vs OOMKilled

$ Terminal

```
kubectl apply -f cpu-hog.yaml  # limit 200m
kubectl top pod cpu-hog
```

OUTPUT

```
cpu-hog   200m   0Mi
# held exactly at the limit
```

$ Terminal

```
kubectl exec cpu-hog -- grep \
  throttled /sys/fs/cgroup/cpu.stat
```

OUTPUT

```
nr_periods 418
nr_throttled 418

# throttled in every 100 ms period —
# slow, never killed: RESTARTS 0
```

$ Terminal

```
kubectl apply -f memory-hog.yaml
# memory limit 64Mi
```

OUTPUT

```
memory-hog   0/1   OOMKilled   2 (25s ago)
Reason: OOMKilled   Exit Code: 137
```

Lab: root, safe, and the namespace check

$ Terminal

```
kubectl exec sc-root -- id
kubectl exec sc-root -- \
  sh -c 'echo x > /etc/motd'
```

OUTPUT

```
uid=0(root) gid=0(root) ...
# the write works: it is root
```

$ Terminal

```
kubectl exec sc-safe -- id
kubectl exec sc-safe -- ls -ld /tmp
kubectl exec sc-safe -- touch /hello
```

OUTPUT

```
uid=1000 gid=3000 ...
drwxrwsrwx ... root 2000 ... /tmp
touch: /hello: Read-only file system
```

$ Terminal

```
kubectl -n sc-lab apply -f sc-root.yaml
# namespace enforces the restricted level
```

OUTPUT

```
Forbidden: violates PodSecurity
"restricted:latest" ...
```

Lab: one Pod, three probes — and a Service

$ Terminal

```
kubectl apply -f probes-pod.yaml
timeout 90 kubectl get pod probe-demo -w
```

OUTPUT

```
0/1   0   1s   ← startup probe
1/1   0   35s  ← ready
```

$ Terminal

```
# the app deletes /tmp/ready at 60 s
kubectl get events … | grep -E \
  'Unhealthy|Killing'
```

OUTPUT

```
0/1   0           65s   ← readiness
0/1   1 (1s ago)  81s   ← liveness

Killing  Container my-app failed
         liveness probe, will be restarted
```

$ Terminal

```
kubectl exec <web-pod> -- rm \
  /usr/share/nginx/html/index.html
```

OUTPUT

```
web-…-6cwbc  ready=false  RESTARTS 0
web-…-pwdvr  ready=true   6 × 200 OK
```

Lab: node affinity — required, preferred, NotIn

$ Terminal

```
kubectl apply -f node-affinity-pod.yaml
# requires storage In [ssd]
```

OUTPUT

```
node-affinity-pod   0/1   Pending
2 node(s) didn't match Pod's
  node affinity/selector
```

$ Terminal

```
kubectl label node lab-worker2 storage=ssd
kubectl label node lab-worker storage=hdd
```

OUTPUT

```
node-affinity-pod   Running   lab-worker2
# remove the label: the Pod keeps running
```

$ Terminal

```
kubectl cordon lab-worker2
kubectl scale deploy prefer-ssd --replicas=6
```

OUTPUT

```
4  lab-worker2   ← preferred
2  lab-worker    ← preference lost, not Pending
```

Lab: pod anti-affinity and topology spread

$ Terminal

```
kubectl apply -f \
  db-anti-affinity.yaml
```

OUTPUT

```
db-…-6xf8c   Running   lab-worker2
db-…-fbx5m   Running   lab-worker
db-…-qkbdd   Pending   ← one per node, 2 workers
```

$ Terminal

```
kubectl apply -f spread.yaml
# maxSkew: 1
```

OUTPUT

```
4 Pending   1 lab-worker   1 lab-worker2
didn't match pod topology spread constraints

# the tainted control plane counts
# as a domain with 0 Pods
```

$ Terminal

```
kubectl patch … \
  nodeTaintsPolicy: Honor
```

OUTPUT

```
3 Running   lab-worker
3 Running   lab-worker2
```

Lab: NoSchedule vs NoExecute

$ Terminal

```
kubectl taint nodes lab-worker2 \
  dedicated=ops:NoSchedule
kubectl apply -f taint-pods.yaml
```

OUTPUT

```
normal-pod   Pending   ← no toleration
ops-pod, ops-agent, ops-grace   Running
```

$ Terminal

```
kubectl taint nodes lab-worker2 \
  dedicated=ops:NoExecute
```

OUTPUT

```
ops-pod     evicted  ← NoSchedule toleration only
ops-agent   stays    ← empty effect = every effect
ops-grace   evicted after tolerationSeconds: 30
```

$ Terminal

```
kubectl taint nodes lab-worker2 dedicated-
```

OUTPUT

```
node/lab-worker2 untainted
Taints: <none>  → normal-pod Running
```

A NoSchedule toleration doesn't cover NoExecute. Tolerate each effect you expect, or leave effect empty to match all.

Scheduling & reliability in 2026 — what changed

In-Place Pod Resize — GA in v1.35

CPU and memory requests/limits can be changed for a running container through the Pod resize subresource, potentially avoiding Pod recreation. Runtime support and resize policy still matter.

DRA — device-level scheduling

Dynamic Resource Allocation extends scheduling to complex hardware. Kubernetes v1.37 made DRA device taints and tolerations stable, applying the familiar repel / tolerate model to individual devices.

2026 mental model **Node taints protect Nodes • Device taints protect individual allocated hardware • In-place resize changes CPU/memory allocation without the old delete-and-recreate assumption.**

Verified on the v1.37 cluster: InPlacePodVerticalScaling and DRADeviceTaints report no alpha/beta stage — both GA.

Spread and share — 2026 scheduling practice

Topology spread constraints

maxSkew: 1 keeps replica counts per node or zone within one of each other — softer than anti-affinity, and many Pods per node are fine.

Add nodeTaintsPolicy: Honor (stable) so tainted nodes, like the control plane, don't count as empty domains.

Pod-level resources — beta in v1.37

spec.resources on the Pod sets one CPU and memory budget that all its containers share — no guessing how to split it between app and sidecar.

The scheduler books the Pod-level request. On by default in v1.37 (PodLevelResources, beta).

2026 takeaway: spread with constraints, budget per Pod where it fits, and read Events whenever a Pod is Pending.

Common pitfalls — diagnose the symptom

Pending Pod

Unmatched affinity / anti-affinity or taint.

Inspect Events, node labels, taints and tolerations.

OOMKilled

Memory limit too low or workload spikes.

kubectl describe pod → state / last state / events.

Readiness fails

Pod may receive no Service traffic without restarting.

Check probe path, dependencies and endpoint state.

Probe flapping

Timing too aggressive.

Add a startupProbe for slow boots; tune period, timeout, thresholds.

Affinity too strict

Required rule or spread has no eligible node.

Fix labels, relax to preferred, or set nodeTaintsPolicy: Honor.

Toleration confusion

Toleration permits; it does not force placement.

Pair with affinity when you need dedicated nodes.

Module recap — resource, health, placement

Requests
Scheduling floor

Limits
Runtime ceiling

Liveness
Restart unhealthy container

Readiness
Remove from traffic

Startup
Protect slow boot

Affinity
Pods choose Nodes

Anti-Affinity
Avoid matching set

Taints
Nodes repel Pods

Tolerations
Pods may enter

Pod Affinity
Co-locate Pods

Pod Anti-Affinity
Spread Pods

Observe
get / describe / top / events

Reliable workloads need four answers: “How much?”, “Are you healthy?”, “What may you do?”, and “Where should you run?”

KNOWLEDGE CHECK

9 questions

Before you move on



Real resource, probe and scheduling problems, with the terminal output you'd see. Answer key on the last quiz slide.

Question 1 of 9

A new Pod is Pending, yet kubectl top shows the nodes almost idle. What's going on?

$ Scenario

```
k describe pod api-7d9 | tail -1
k top nodes
```

OUTPUT

```
0/2 nodes are available: 2 Insufficient cpu.

NAME     CPU(cores)   CPU(%)
node-1   410m         10%
node-2   380m         9%
```

Think about

What does the scheduler add up?

CHOOSE ONE

A. The kubelet is broken — restart it
B. Scheduling uses requests, not live usage — existing requests already fill the nodes
C. kubectl top is wrong — reinstall metrics-server
D. The Pod's CPU limit is too low

Question 2 of 9

This container keeps restarting. What's the right fix?

$ Scenario

```
k describe pod report-5c8 | grep -A3 'Last State'
```

OUTPUT

```
Last State:  Terminated
  Reason:    OOMKilled
  Exit Code: 137
```

Think about

Which resource kills?

CHOOSE ONE

A. Raise the CPU limit
B. Add a readiness probe
C. Raise the memory limit to fit real usage — or fix the leak
D. Add a toleration

Question 3 of 9

Users say the API is slow. There are no restarts. What is most likely happening?

$ Scenario

```
k top pod api-6b7
k exec api-6b7 -- grep throttled
/sys/fs/cgroup/cpu.stat
```

OUTPUT

```
api-6b7   500m   210Mi     # CPU limit: 500m
nr_throttled 9312
```

Think about

CPU at exactly the limit

CHOOSE ONE

A. The container is CPU-throttled at its limit — raise or remove the CPU limit
B. The container is being OOMKilled
C. The liveness probe is failing
D. The node has a NoSchedule taint

Question 4 of 9

A Pod is Running with 0 restarts, but the Service never sends it traffic. Where do you look?

$ Scenario

```
k get pod web-7f9-kq2
k describe pod web-7f9-kq2 | grep
Unhealthy
```

OUTPUT

```
web-7f9-kq2   0/1   Running   0   9m
Readiness probe failed: HTTP probe failed
  with statuscode: 503
```

Think about

READY vs RESTARTS

CHOOSE ONE

A. Delete the Service and create it again
B. Increase the liveness failureThreshold
C. Give the Pod more memory
D. The readiness probe fails — fix what it checks; it won't restart the Pod

Question 5 of 9

An app needs 90 seconds to boot. The liveness probe kills it before it's ready, over and over. Best fix?

$ Scenario

```
k get pod legacy-4d2
```

OUTPUT

```
legacy-4d2   0/1   CrashLoopBackOff   6
Liveness probe failed …
failed liveness probe, will be restarted
```

Think about

Which probe protects slow starts?

CHOOSE ONE

A. Remove the liveness probe
B. Add a startupProbe with periodSeconds × failureThreshold > 90 s
C. Set restartPolicy: Never
D. Use a readiness probe instead of liveness

Question 6 of 9

A Pod with a required node affinity for storage=ssd stays Pending. What do you do?

$ Scenario

```
k get events --field-selector
reason=FailedScheduling
```

OUTPUT

```
0/3 nodes are available: 1 node(s) had untolerated
taint(s), 2 node(s) didn't match Pod's node
affinity/selector.
```

Think about

Read the reason literally

CHOOSE ONE

A. Label a node storage=ssd — or relax the rule to preferred
B. Add a toleration for the control-plane taint
C. Increase the Pod's CPU request
D. Restart the scheduler

Question 7 of 9

GPU nodes are tainted gpu=true:NoSchedule. The ML Pods tolerate it, but they keep landing on normal nodes. Why?

$ Scenario

```
k get pods -l app=trainer -o wide
```

OUTPUT

```
trainer-0   Running   node-cpu-3
trainer-1   Running   node-cpu-1
```

Think about

Allow vs attract

CHOOSE ONE

A. The taint effect should be NoExecute
B. The tolerations are wrong
C. A toleration only allows a node — add node affinity (or nodeSelector) for the GPU nodes
D. GPU nodes need a PriorityClass

Question 8 of 9

You added dedicated=ops:NoExecute to a node. A Pod that tolerates dedicated=ops:NoSchedule was evicted. Why?

$ Scenario

```
k get events --field-selector
reason=TaintManagerEviction
```

OUTPUT

```
ops-pod   Marking for deletion Pod default/ops-pod
```

Think about

Effects are matched one by one

CHOOSE ONE

A. NoExecute evicts every Pod, tolerated or not
B. The node was drained
C. The toleration value was wrong
D. The toleration covers NoSchedule only — add one for NoExecute (or leave effect empty)

Question 9 of 9

6 replicas, spread with maxSkew: 1 per node, on 2 workers + a tainted control plane. Four stay Pending. Fix?

$ Scenario

```
k get pods -l app=spread --no-headers |
awk '{print $3}' | sort | uniq -c
```

OUTPUT

```
      4 Pending
      2 Running
didn't match pod topology spread constraints
```

Think about

Which nodes count as domains?

CHOOSE ONE

A. Set maxSkew: 0
B. Add nodeTaintsPolicy: Honor so the tainted node isn't counted
C. Remove the control-plane taint
D. Use hostNetwork: true

Answer key

Score yourself

9 questions
1 point each

ANSWERS

Q1	B	requests are booked, not usage	→ slide 38 • Step 6
Q2	C	memory limit or leak	→ slide 39 • Step 5
Q3	A	CPU throttling	→ slide 39 • Step 4
Q4	D	readiness probe fails	→ slide 41 • Step 11
Q5	B	startupProbe	→ slide 17 • Step 10
Q6	A	label a node or prefer	→ slide 42 • Step 12
Q7	C	toleration + node affinity	→ slide 33 • Step 16
Q8	D	tolerate NoExecute too	→ slide 44 • Step 17
Q9	B	nodeTaintsPolicy: Honor	→ slide 43 • Step 15

8–9	Module 8 ready
6–7	re-read misses
≤ 5	redo Steps 2–17

MODULE 7 COMPLETE

Next: Module 8

Storage — PV, PVC, StorageClass & StatefulSets



Resources define the budget. Probes define health. Scheduling rules define placement.